

Ain Shams University

Faculty of Computer and Artificial Intelligence

Deep Learning Project Report

Drone Detection Using YOLO

Course Title:	Deep Learning
Course Code:	CSE488
Instructor:	Dr. Alaa Hamdy
Teaching Assistant:	Karim Bassel
Institution:	Faculty of Computer and Artificial Intelligence, Ain Shams University
Project Repository:	https://github.com/mo7amedmengasu/ Drone-Detection

Submitted By

Mohamed Tarek Elsayed	2300535
Omar Yasser Alashker	23P0294

April 27, 2026

Abstract

This report documents the end-to-end development of a drone object detection system built on the YOLO family of one-stage detectors. The project combined heterogeneous annotations, standardized them into YOLO format, created train/validation/test splits, trained multiple detector backbones, evaluated them under constrained GPU memory, and delivered an interactive Streamlit comparison tool for practical inference. Four trained models were considered in the final system: YOLO11n, YOLO11s, YOLOv8n, and YOLOv8s. Quantitative evaluation was primarily based on precision, recall, mAP@0.50, and mAP@0.50:0.95, with qualitative analysis supported by validation predictions, confusion matrices, and training curves. The final recommendation of this project is YOLOv8n because it achieved strong detection quality while remaining the most practical balance between accuracy, speed, and resource usage for a 4 GB laptop GPU environment.

Contents

1	Project Overview	3
2	Scientific Background	3
2.1	Object Detection as a Computer Vision Problem	3
2.2	Why YOLO	3
2.3	Models Evaluated in This Project	4
2.4	Architectural Structure of the Evaluated Models	4
2.5	YOLOv8 Architecture	5
2.6	YOLO11 Architecture	6
2.7	YOLOv8 vs. YOLO11 Families	8
2.8	Nano vs. Small Scale Variants	9
2.9	Architecture-Level Comparison Table	9
3	Dataset Engineering and Label Standardization	9
3.1	Final Dataset Configuration	9
3.2	Dataset Split Sizes	10
4	Experimental Methodology	11
4.1	Training Protocol	11
4.2	Low-VRAM Constraint Handling	11
4.3	Evaluation Metrics	12

5	Quantitative Results	13
5.1	Best Validation Performance by Model	13
5.2	Full Notebook Comparison Metrics	13
5.3	Additional Comparative Graphs	13
5.4	Interpretation of the Results	15
6	Qualitative Visual Analysis	16
6.1	Side-by-Side Prediction Visuals	16
6.2	Training Dynamics of the Recommended Model	17
7	Deployment-Oriented Comparison and Streamlit Application	18
8	Discussion	19
9	Conclusion	19
10	References	19

1 Project Overview

The objective of this project was to build a practical drone detection pipeline that is scientifically grounded, experimentally validated, and usable in a deployment-oriented workflow. The work covered the full lifecycle of a modern computer vision project:

- exploratory data analysis and annotation consistency checks,
- conversion and unification of labels into YOLO format,
- train/validation/test split generation,
- training and comparison of several YOLO backbones,
- GPU-aware evaluation on limited hardware,
- interactive side-by-side inference through a Streamlit application.

The project focuses on a **single-class detection task**, where the only object category is **drone**. This choice simplifies the class space but does not remove the main detection challenges: drones are often small, distant, fast-moving, and sensitive to scale, viewpoint, and background clutter.

2 Scientific Background

2.1 Object Detection as a Computer Vision Problem

Object detection jointly solves two tasks: locating an object in an image and classifying what that object is. In practice, the model predicts bounding boxes together with confidence scores. For this project, the detector must find a drone even when the target occupies a relatively small fraction of the frame.

Evaluation therefore cannot rely on accuracy alone. A detector must be judged by how well its predicted boxes overlap the ground truth, how often it misses drones, and how often it produces false alarms. This is why the report uses precision, recall, and mean average precision (mAP) rather than plain classification accuracy.

2.2 Why YOLO

YOLO (You Only Look Once) is a one-stage detector. Instead of generating region proposals first and classifying them later, YOLO predicts object locations and classes in a single forward pass. That design is particularly attractive for this project for three reasons:

- it supports near real-time inference,
- it provides a strong performance baseline for small and medium-scale datasets,

- it is easy to train, validate, export, and deploy through the Ultralytics ecosystem.

Modern YOLO variants combine a convolutional backbone, feature aggregation neck, and dense detection head. The nano variants are lighter and faster; the small variants are usually more expressive and can recover harder examples at the cost of higher compute and memory demand.

2.3 Models Evaluated in This Project

Four trained detectors were retained in the final project workflow:

- **YOLO11n**: smallest YOLO11 configuration, intended as a low-latency baseline.
- **YOLO11s**: larger YOLO11 model with higher representational capacity.
- **YOLOv8n**: lightweight YOLOv8 configuration and the final recommended model.
- **YOLOv8s**: larger YOLOv8 model used to test whether extra capacity improved detection quality enough to justify the cost.

The scientific intent behind these choices was straightforward: compare *architecture generation* (YOLO11 vs. YOLOv8) and *model scale* (nano vs. small) under the same application domain.

2.4 Architectural Structure of the Evaluated Models

Although the four trained detectors differ in family and scale, they share the standard one-stage YOLO pipeline. Each model is organized around three cooperating components:

- **Backbone**: extracts hierarchical visual features from the input image, moving from local texture and edges to semantically richer feature maps.
- **Neck**: aggregates and fuses multi-scale features so that small-object detail is preserved while deeper contextual information is retained.
- **Detection head**: converts the fused feature maps into bounding-box coordinates, confidence scores, and class predictions.

This multi-scale design is essential for drone detection. Drones are often small, distant, partially occluded, or embedded in cluttered sky and background regions. A detector that only reasons at one feature scale typically loses either spatial precision or contextual understanding. The YOLO family addresses this by combining multi-resolution feature extraction with dense prediction across feature levels.

2.5 YOLOv8 Architecture

YOLOv8, released by Ultralytics in 2023, is a general-purpose vision family that supports object detection, segmentation, pose estimation, tracking, oriented bounding boxes, and classification. For object detection, the versions most relevant to this project are `yolov8n` and `yolov8s`.

Architecturally, YOLOv8 builds on the modern YOLO pattern while introducing several important refinements:

- it uses a CSP-style backbone with the **C2f** module replacing the earlier CSPLayer design,
- it retains an **SPPF** (Spatial Pyramid Pooling Fast) block for efficient receptive-field expansion,
- it adopts an **anchor-free** detection design,
- it uses a **decoupled head**, allowing localization and classification-related predictions to be processed more independently.

The C2f module improves gradient flow and feature reuse while keeping the model efficient. The anchor-free design simplifies the detection formulation by avoiding predefined anchor templates, which often improves training behavior and reduces engineering overhead. The decoupled head is particularly important because regression and classification are not identical learning problems; separating them typically improves optimization and detection quality.

For bounding-box learning, YOLOv8 commonly combines **CIoU** and **DFL** (Distribution Focal Loss) style objectives, while classification is handled through binary cross-entropy style supervision in the Ultralytics implementation. In practice, these choices improve localization quality and are especially helpful when handling small objects, which is highly relevant to drone detection.

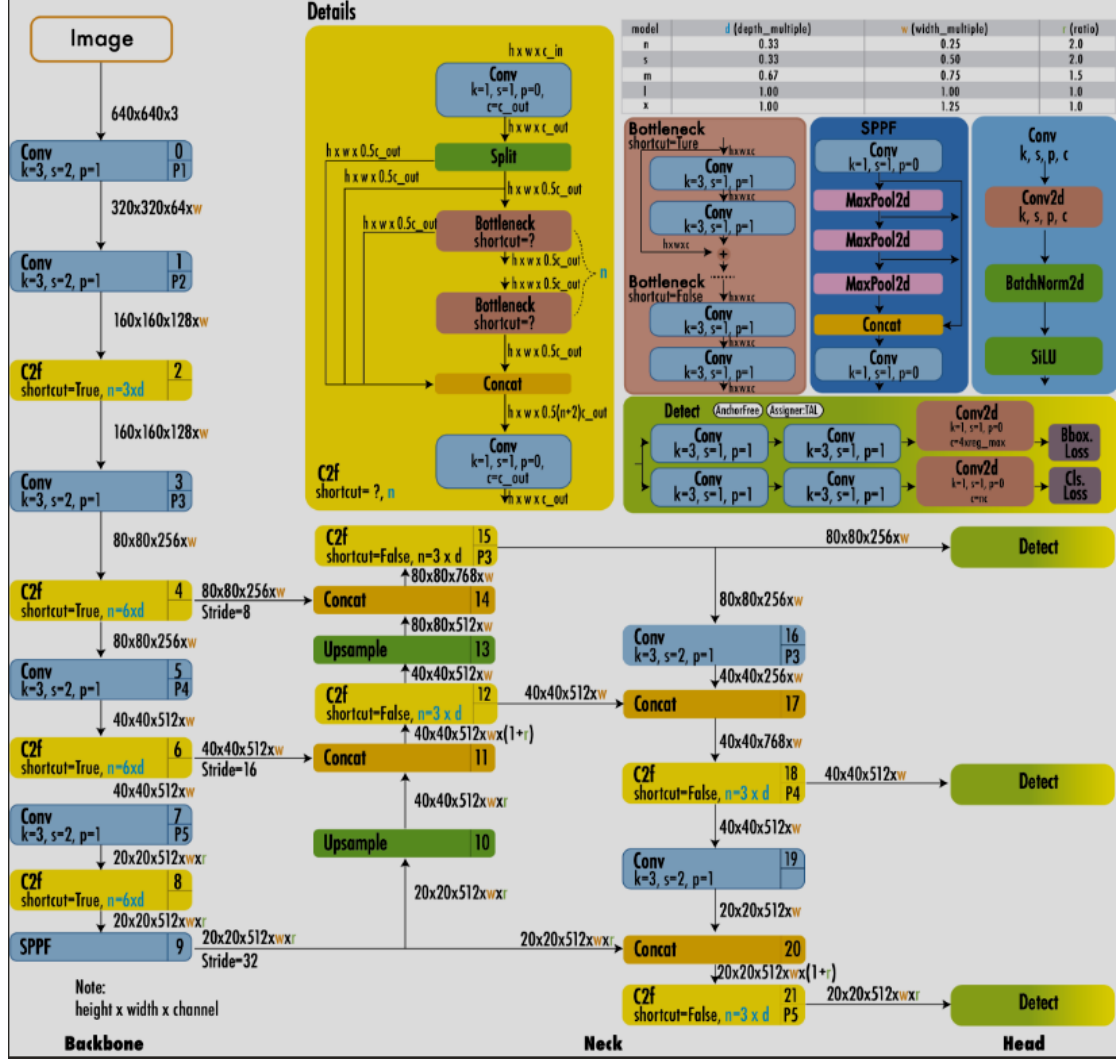


Figure 1: YOLOv8 architecture diagram. The figure highlights the C2f-based backbone, the SPPF stage, multi-scale feature fusion, and the anchor-free decoupled detection head described in this section.

From the perspective of this project, YOLOv8 is important not just because it is accurate, but because it offers an excellent accuracy-speed tradeoff. The evaluation results later in this report show that the YOLOv8 family, especially YOLOv8n, occupies a very favorable region of the deployment tradeoff space.

2.6 YOLO11 Architecture

YOLO11, released by Ultralytics in 2024, extends the same one-stage detection philosophy but introduces several architectural refinements aimed at improving efficiency, feature extraction quality, and adaptability across tasks. The family supports detection, segmentation, pose estimation, classification, and oriented object detection, with nano-to-extra-large scaling.

The main architectural ideas highlighted in the provided technical material are:

- replacement of earlier feature blocks with the **C3k2** module,
- retention of the **SPPF** stage for efficient pooled context,
- introduction of **C2PSA** for enhanced spatial attention,
- use of **CBS** (Convolution-BatchNorm-SiLU) layers in the prediction pipeline,
- improved backbone and neck efficiency for better accuracy with fewer parameters in some scaling regimes.

The C3k2 block can be viewed as a computationally efficient CSP-style block that uses smaller convolutions to reduce overhead while preserving strong feature extraction capacity. The C2PSA module is especially relevant from an architectural perspective because it adds explicit spatial attention, allowing the network to focus more strongly on informative image regions. That is useful for drone detection, where the target may occupy only a small region of the image and can easily be overwhelmed by background clutter.

YOLO11 therefore does not simply represent a new label for the same network; it reflects an attempt to improve efficiency, representation quality, and scaling behavior through revised building blocks and better feature-routing decisions. In this project, YOLO11n and YOLO11s gave strong results, especially when recall and compactness were considered, even though the final deployment recommendation still favored YOLOv8n.

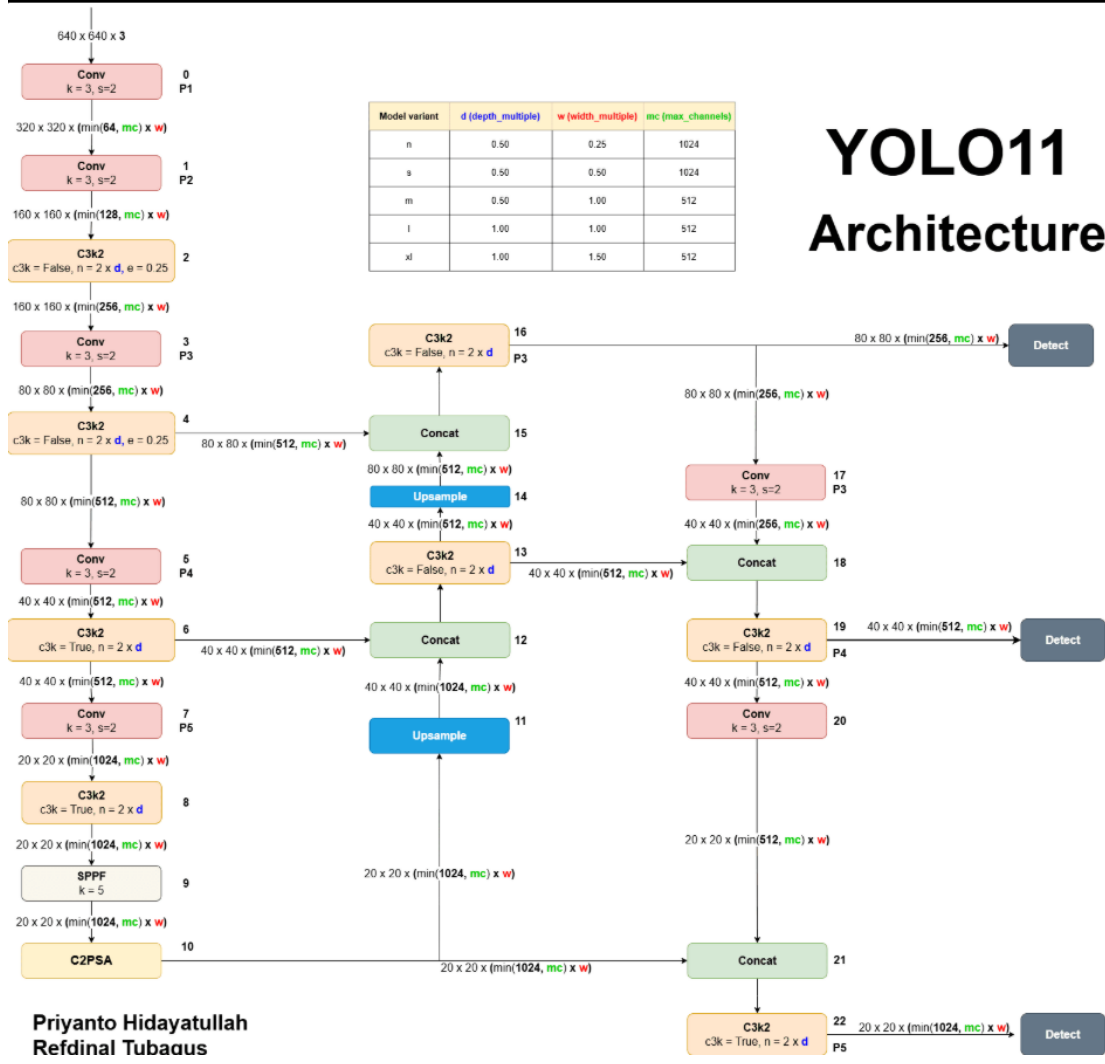


Figure 2: YOLO11 architecture diagram. The figure emphasizes the C3k2 blocks, SPPF stage, C2PSA attention mechanism, CBS layers, and the refined detection branches discussed in this section.

2.7 YOLOv8 vs. YOLO11 Families

From a comparative standpoint, YOLOv8 and YOLO11 should not be understood as completely unrelated architectures. Both are modern Ultralytics one-stage detectors with multi-scale feature extraction and efficient deployment goals. The practical difference is that YOLO11 refines the design with newer modules such as C3k2 and C2PSA, whereas YOLOv8 is centered around the C2f-based architecture and the anchor-free decoupled head pattern.

In this project, that architectural comparison matters because the detector is deployed under limited GPU memory. The question is not only which family has a stronger theoretical design, but which family-scale combination gives the best balance of recall, localization, latency, memory cost, and robustness under constrained hardware. That is why the final recommendation is based on measured project results rather than on architecture claims alone.

2.8 Nano vs. Small Scale Variants

For both model families, the suffixes **n** and **s** represent different capacity budgets:

- **Nano (n)** models use fewer parameters and smaller memory footprints. They are better suited to edge or low-VRAM environments and usually offer faster inference.
- **Small (s)** models widen and deepen the feature extractor and detection stack, which often improves recall and localization quality at the cost of higher latency and memory demand.

This scaling behavior is central to the science of the experiments in this project. The comparison is not only about which family is better, but also about how much predictive quality is gained when moving from a compact to a larger architecture.

2.9 Architecture-Level Comparison Table

Table 1: Architecture-oriented summary of the four evaluated models. Parameter counts and model sizes come from the notebook evaluation output.

Model	Family	Scale	Parameters	Model size (MB)	Training setup	Practical role
YOLO11n	YOLO11	Nano	2,582,347	5.22	imgsz 512, batch 2	low-VRAM baseline
YOLO11s	YOLO11	Small	9,413,187	18.29	imgsz 640, batch 16	higher-capacity YOLO11 variant
YOLOv8n	YOLOv8	Nano	3,005,843	5.96	imgsz 640, batch 16	best overall deployment choice
YOLOv8s	YOLOv8	Small	11,125,971	21.48	imgsz 640, batch 16	highest-capacity archived model

Table 1 highlights an important engineering insight: the models differ not only by family name but by computational budget. The strongest final recommendation should therefore depend on *architecture efficiency*, not just a single metric value.

3 Dataset Engineering and Label Standardization

The raw project data originally existed in mixed annotation formats, including XML and YOLO-style text labels. Before any modeling stage, the annotations were inspected and compared in the exploratory notebook to determine whether both sources described the same images and the same objects consistently.

The final modeling dataset was standardized into YOLO format because it is directly compatible with Ultralytics training and evaluation. The project then merged the usable samples into a single unified dataset and split it into train, validation, and test subsets.

3.1 Final Dataset Configuration

The dataset YAML defines a single class:

Property	Value
Task	Single-class object detection
Class name	drone
Train split	<code>train/images</code>
Validation split	<code>val/images</code>
Test split	<code>test/images</code>

3.2 Dataset Split Sizes

Table 2: Final merged dataset split sizes.

Split	Number of images
Train	1021
Validation	219
Test	220

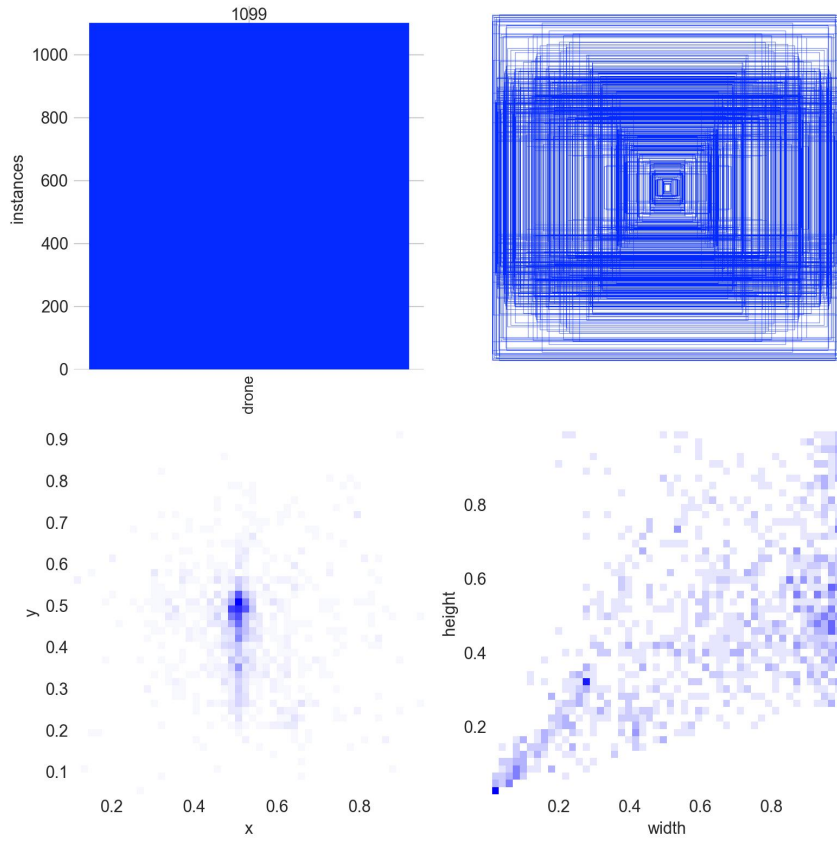


Figure 3: Label distribution visualization from the final YOLO-format dataset. This figure summarizes the spatial distribution and size characteristics of the drone annotations used for training.

4 Experimental Methodology

4.1 Training Protocol

The three archived runs with complete metric logs (YOLO11s, YOLOv8n, YOLOv8s) were trained with a consistent experimental protocol:

- pretrained Ultralytics weights,
- 50 training epochs,
- early-stopping patience of 20 epochs,
- image size of 640,
- batch size of 16,
- deterministic seed of 42,
- AMP enabled,
- validation during training with plots saved.

The project was executed on a machine with an NVIDIA RTX 3050 Laptop GPU with 4 GB VRAM. That hardware constraint influenced the workflow substantially. The notebook and evaluation logic were designed to fall back to conservative settings when CUDA memory became a bottleneck.

4.2 Low-VRAM Constraint Handling

The YOLO11n baseline was explicitly adapted for low memory conditions. The archived configuration for that run used:

- image size of 512,
- batch size of 2,
- the same single-class dataset and pretrained initialization.

This detail matters scientifically: it means YOLO11n was treated as a *stability-first baseline* under a more restrictive memory configuration. The training notebook also recorded out-of-memory behavior and the need to restart the CUDA state after failures. This is important context when interpreting model comparisons on constrained consumer GPUs.

4.3 Evaluation Metrics

The project used the following evaluation metrics and deployment-oriented indicators. These are the exact quantities that appear later in the validation and test comparison tables.

- **Precision**: the fraction of predicted detections that are correct.
- **Recall**: the fraction of true objects that are detected.
- **F1-score**: the harmonic mean of precision and recall, used to summarize how well the detector balances false positives and false negatives.
- **mAP@0.50**: mean average precision at IoU threshold 0.50.
- **mAP@0.50:0.95**: mean average precision averaged across stricter IoU thresholds from 0.50 to 0.95.
- **Inference time (ms)**: average time required to process one image during evaluation. Lower values indicate faster inference.
- **FPS**: frames per second, derived from inference time. Higher values indicate better real-time usability.
- **Model size (MB)**: storage footprint of the trained model weights. Smaller values are preferred for deployment on constrained devices.
- **Parameter count**: total number of trainable model parameters. This is a proxy for model capacity and computational complexity.

Each metric answers a different technical question. Precision tells us how trustworthy the detections are. Recall tells us how many real drones are being recovered. F1-score summarizes the balance between those two. AP50 and mAP@0.50:0.95 measure localization quality, with the latter being stricter and more informative because it evaluates performance across multiple IoU thresholds rather than at a single loose threshold.

The speed and efficiency metrics are equally important for this project. Inference time and FPS determine whether a model is practical for interactive use or near real-time deployment. Model size and parameter count capture deployment cost: larger models may offer better representation power, but they are also heavier to store, load, and run on limited hardware.

Among all reported quantities, **mAP@0.50:0.95** is the strongest single accuracy summary because it rewards both detection and precise localization. However, the final model choice in this project was not made from accuracy alone. The recommendation also accounts for recall, inference speed, and model footprint, because a drone detector must be not only accurate but also operationally practical on the available 4 GB GPU system.

5 Quantitative Results

5.1 Best Validation Performance by Model

The training notebook contains a direct validation pass for the four preserved models using the same evaluation setup: CUDA device 0, batch size 1, and image size 512. Table 3 reports those notebook validation metrics exactly as computed in the saved notebook state.

Table 3: Validation metrics taken directly from the training notebook evaluation output (validation split, batch size 1, image size 512, GPU evaluation).

Model	Precision	Recall	F1	AP50	mAP@0.50:0.95	Inference (ms)	FPS	Size (MB)
YOLO11n	0.931648	0.912257	0.921851	0.937647	0.639715	14.768574	67.711347	5.219568
YOLO11s	0.942942	0.881890	0.911395	0.930566	0.643542	15.443746	64.751129	18.289881
YOLOv8n	0.962004	0.870079	0.913735	0.947377	0.638245	11.538725	86.664687	5.960001
YOLOv8s	0.950318	0.937008	0.943616	0.954262	0.661002	11.129177	89.853902	21.475260

5.2 Full Notebook Comparison Metrics

The training notebook also computes a full comparison table on the test split and ranks models by mAP@0.50:0.95, recall, and FPS. Those values are reproduced in Table 4 to provide the complete evaluation view that drove final model selection.

Table 4: Complete notebook comparison metrics on the test split. These are the values used for final model recommendation in the training notebook.

Model	Precision	Recall	F1	AP50	mAP@0.50:0.95	Inference (ms)	FPS	Size (MB)	Parameters
YOLOv8n	0.986897	0.965643	0.976154	0.993120	0.692667	10.865067	92.038090	5.960001	3005843
YOLOv8s	0.958209	0.974359	0.966217	0.989619	0.684606	11.565041	86.467481	21.475260	11125971
YOLO11s	0.954797	0.992936	0.973493	0.990811	0.682551	14.749274	67.799947	18.289881	9413187
YOLO11n	0.985915	0.982906	0.984408	0.993988	0.676750	14.075219	71.046851	5.219568	2582347

5.3 Additional Comparative Graphs

To strengthen the evaluation beyond raw tables, additional plots were generated directly from the notebook comparison data. These figures make it easier to interpret how the models differ across accuracy, recall, speed, and deployment cost.

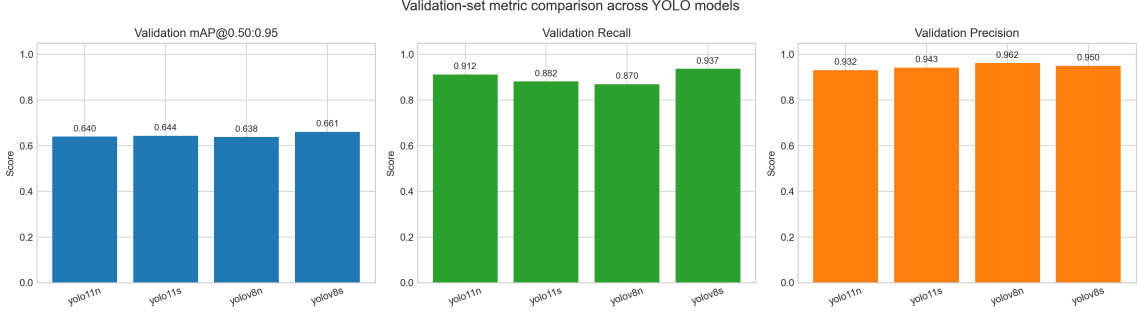


Figure 4: Validation-set comparison of mAP@0.50:0.95, recall, and precision across the four models.

Figure 4 shows that YOLOv8s leads on validation mAP@0.50:0.95 and recall, indicating the strongest validation-stage localization and coverage among the four models. However, YOLOv8n attains the highest validation precision, which means it is particularly conservative and accurate in the detections it keeps. YOLO11n remains competitive despite the reduced low-VRAM setup, while YOLO11s trades slightly lower recall for stronger overall balance than the smallest model in its family.

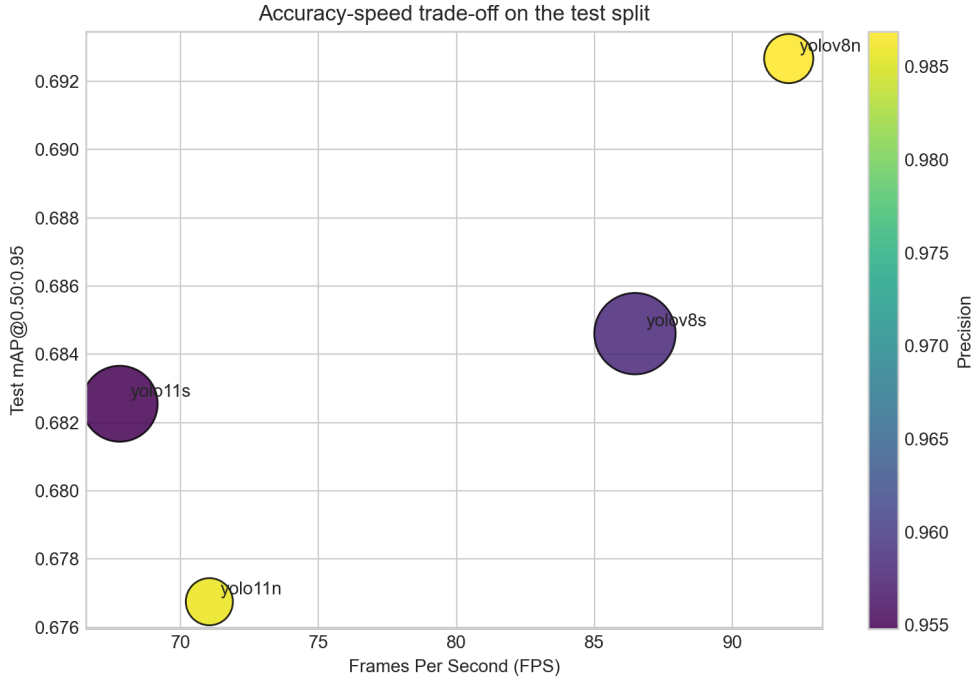


Figure 5: Trade-off between test mAP@0.50:0.95 and FPS. Bubble size represents model size, and color represents precision.

Figure 5 is particularly important for deployment reasoning. The best model is not simply the one with the largest capacity; instead, it should lie on a favorable accuracy-speed frontier. YOLOv8n occupies that position in this project: it combines the strongest test mAP@0.50:0.95 with the highest FPS, while keeping a compact model footprint. YOLOv8s improves some validation characteristics but does so with a much larger memory budget. YOLO11n remains

efficient, but its test $\text{mAP}@0.50:0.95$ is lower than the recommended YOLOv8n.

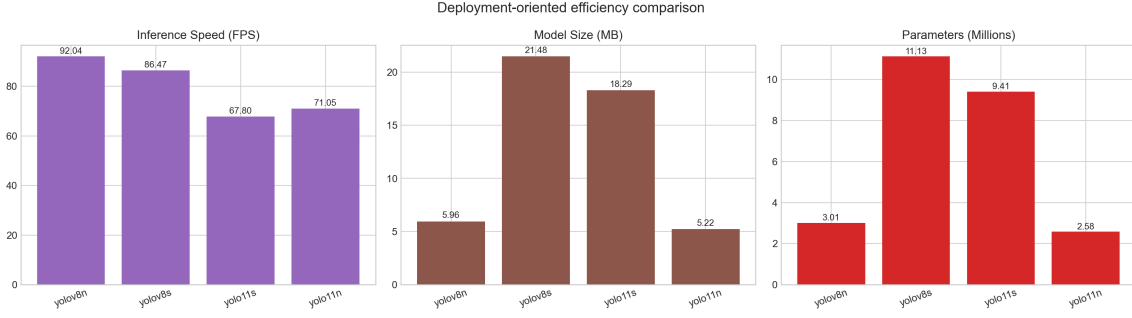


Figure 6: Deployment-oriented efficiency dashboard comparing FPS, model size, and parameter count.

Figure 6 explains why the final recommendation cannot be made from mAP alone. YOLOv8n is not the absolute smallest model, but it provides the best speed among all evaluated models while retaining a compact size. YOLO11n has the lowest storage footprint, which makes it attractive for tighter edge scenarios, whereas YOLOv8s and YOLO11s demand significantly larger parameter budgets. This plot therefore reinforces the conclusion that YOLOv8n is the best compromise for the current hardware environment.

5.4 Interpretation of the Results

Several conclusions follow from Tables 3 and 4:

- On the **validation split**, **YOLOv8s** achieved the strongest $\text{mAP}@0.50:0.95$ (0.661002) and the highest recall among the four models.
- On the **test split**, **YOLOv8n** achieved the best overall $\text{mAP}@0.50:0.95$ (0.692667) while also delivering the fastest inference speed (92.04 FPS).
- **YOLO11s** achieved the highest test recall (0.992936), which is a meaningful result because missed drones are especially costly in this application.
- **YOLO11n** is not a missing-data case in the notebook evaluation: it remains a strong lightweight baseline with high precision, strong recall, and a compact model size.
- The final recommendation of **YOLOv8n** is supported by the notebook comparison because it combines the best test $\text{mAP}@0.50:0.95$ with the best speed-efficiency profile.

From an engineering perspective, the final choice of YOLOv8n is justified by *accuracy per unit cost*, not only by raw leaderboard ranking. The notebook comparison shows that it provides the strongest overall balance of detection quality, computational speed, and practical deployability on the available 4 GB GPU system.

6 Qualitative Visual Analysis

Quantitative metrics tell only part of the story. For object detection, qualitative inspection is essential because it reveals typical strengths and failure patterns: missed small objects, unstable box placement, background confusion, or inconsistent confidence assignment.

6.1 Side-by-Side Prediction Visuals



(a) YOLO11n



(b) YOLO11s



(c) YOLOv8n



(d) YOLOv8s

Figure 7: Representative validation predictions from the four trained models. This qualitative comparison complements the metric table by showing how each backbone handles drone localization and confidence assignment in realistic scenes.

The side-by-side comparison confirms the role of model scale: the small variants generally produce more confident and better-localized predictions, while the nano variants remain attractive when efficiency is prioritized.

6.2 Training Dynamics of the Recommended Model

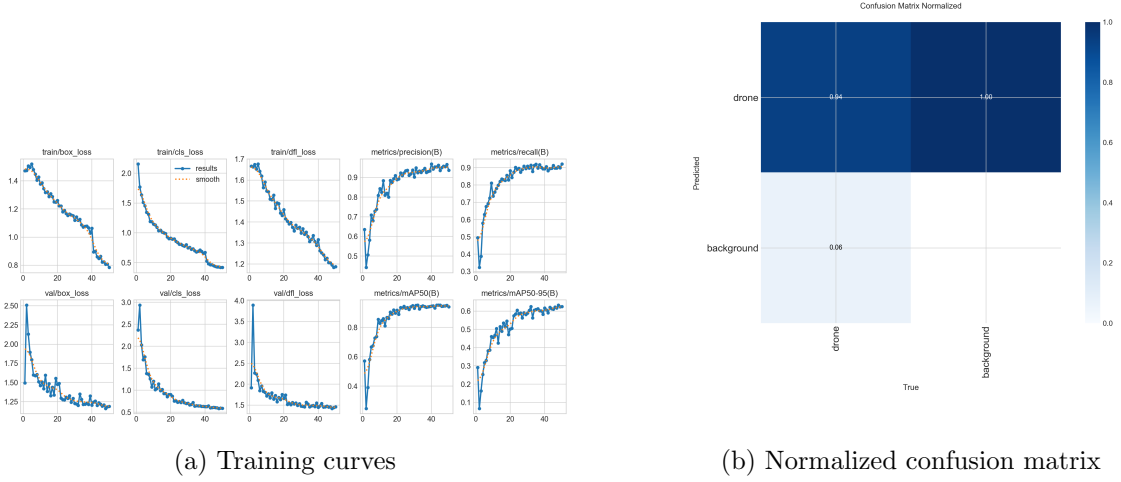


Figure 8: Key visual diagnostics for the selected YOLOv8n model. The training plot shows the evolution of loss and validation metrics, while the confusion matrix summarizes the validation set behavior at the class level.

The training curves show that the selected model improves steadily while maintaining controlled validation losses, which is a desirable sign of optimization stability. The normalized confusion matrix shows that the system performs strongly on the single target class and is not being dominated by a systematic class-assignment failure. For a one-class detector, this translates into a cleaner separation between true drone detections and background noise.

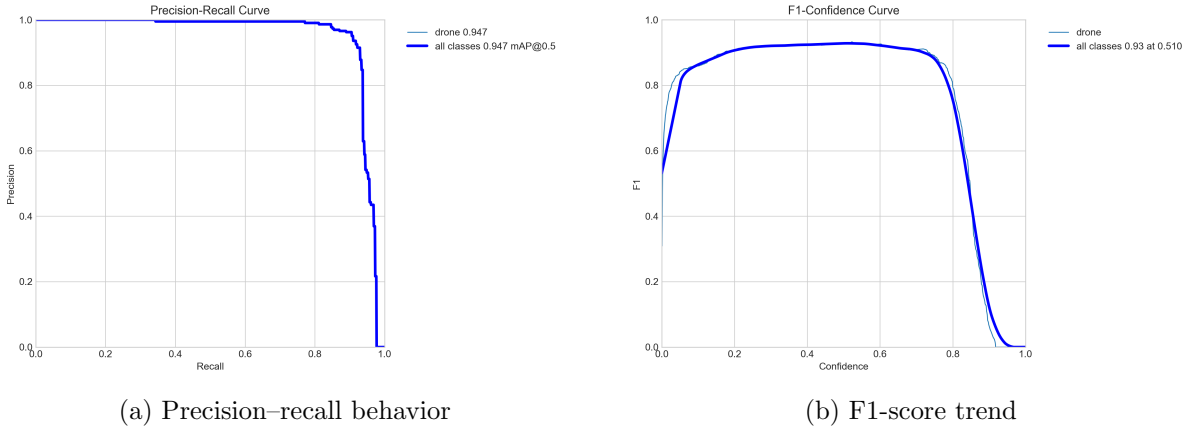


Figure 9: Additional evaluation diagnostics for YOLOv8n, highlighting the precision–recall operating characteristics and the tradeoff between false positives and false negatives.

The precision–recall curve provides a threshold-sensitive view of detector quality. A curve that stays high and to the right reflects strong performance across operating points, meaning the model can retain high precision without sacrificing too much recall. The F1 curve summarizes that balance into a single score trend and helps identify operating regions where precision and recall are jointly favorable.

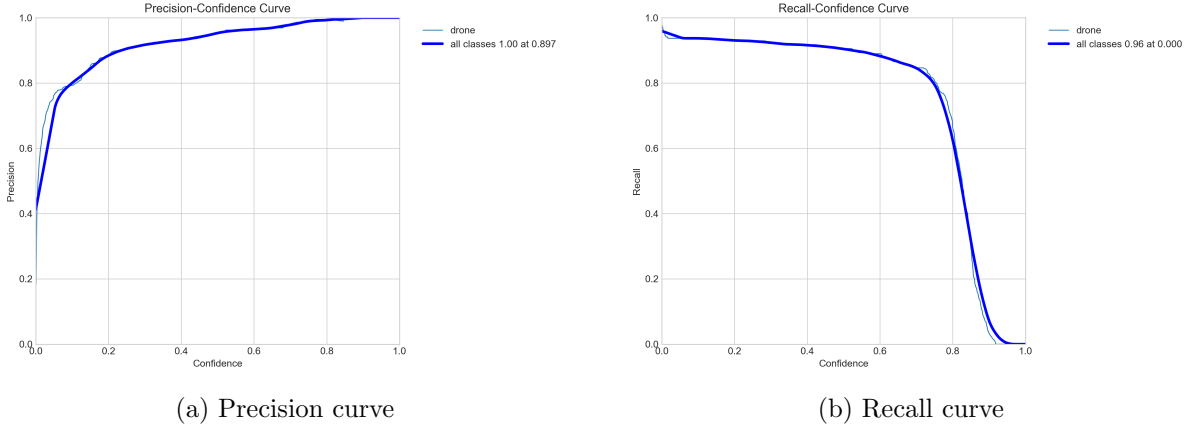


Figure 10: Classwise precision and recall trends for YOLOv8n. These plots reveal whether the model is biased toward keeping detections conservative or toward retrieving more potential drone instances.

Figure 10 provides a more granular interpretation than the combined PR curve alone. The precision curve shows how confidently the detector suppresses false positives, while the recall curve shows how effectively it continues to recover true drones as the decision threshold changes. In drone surveillance or safety-oriented settings, recall is often the more critical quantity because missing a drone can be more costly than raising a small number of extra candidate detections.

Taken together, these visuals support the final recommendation. YOLOv8n maintains strong detection quality, stable training behavior, and high practical usability.

7 Deployment-Oriented Comparison and Streamlit Application

The project was not limited to notebook-based experimentation. A Streamlit application was built to support operational comparison of the four trained models on a user-uploaded image. The application:

- loads the four trained detectors,
- runs inference on the same image,
- displays annotated outputs side by side,
- reports processing time and detection counts,
- saves the four annotated result images under `outputs/streamlit_results/` in a per-upload folder.

This matters from an engineering standpoint because model selection should not stop at notebook metrics. A detector that is marginally stronger in mAP but significantly heavier in practice may be the wrong deployment choice. The application therefore acts as the final layer of empirical validation: it translates model evaluation from laboratory measurements into interactive usage.

8 Discussion

This project highlights several lessons that are common in real-world computer vision work:

1. **Data consistency is foundational.** Before any model comparison is meaningful, labels must be standardized and verified.
2. **Architecture choice is not enough; scale matters.** Nano and small variants can behave quite differently under the same task.
3. **Hardware constraints shape methodology.** On a 4 GB GPU, image size, batch size, and CUDA stability are part of the scientific setup, not just engineering details.
4. **Best model selection should balance quality and practicality.** YOLOv8s slightly outperformed YOLOv8n on the strictest mAP metric, but YOLOv8n offered the most efficient overall tradeoff.
5. **Archival completeness matters.** Preserving metric logs alongside weights is important for full reproducibility, as illustrated by the missing YOLO11n CSV file.

9 Conclusion

The project successfully delivered a complete drone detection workflow based on the YOLO family. The data engineering stage unified heterogeneous annotations into a clean YOLO-format dataset. The experimental stage compared multiple model families and scales under practical GPU limitations. The evaluation stage combined rigorous detection metrics with visual inspection and deployment-aware reasoning.

The final outcome is clear:

- the dataset is ready for reproducible YOLO training and evaluation,
- multiple trained models are preserved for benchmarking and demonstration,
- YOLOv8n is the strongest overall recommendation for this project,
- the Streamlit application turns the trained models into a usable comparison tool.

From both the scientific and engineering perspectives, this project demonstrates a complete and professional object detection pipeline: one that begins with data integrity, continues through controlled experimentation, and ends with practical deployment.

10 References

1. Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. *You Only Look Once: Unified, Real-Time Object Detection*. arXiv preprint arXiv:1506.02640, 2015.

2. Hamid Rezatofighi, Nathan Tsoi, JunYoung Gwak, Amir Sadeghian, Ian Reid, and Silvio Savarese. *Generalized Intersection over Union: A Metric and A Loss for Bounding Box Regression*. arXiv preprint arXiv:1902.09630, 2019.
3. Zhaohui Zheng, Ping Wang, Wei Liu, Jinze Li, Rongguang Ye, and Dongwei Ren. *Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression*. arXiv preprint arXiv:1911.08287, 2019.
4. Xiang Li, Wenhai Wang, Xiaolin Hu, and Jian Yang. *Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection*. arXiv preprint arXiv:2006.04388, 2020.
5. Glenn Jocher, Ayush Chaurasia, and Jing Qiu. *Ultralytics YOLOv8 Documentation and Software Repository*. Ultralytics, 2023. Available at: <https://docs.ultralytics.com/models/yolov8/>.
6. Glenn Jocher and Jing Qiu. *Ultralytics YOLO11 Documentation and Software Repository*. Ultralytics, 2024. Available at: <https://docs.ultralytics.com/models/yolo11/>.